TP Noté : Création d’un capteur de temperature et d’humidité



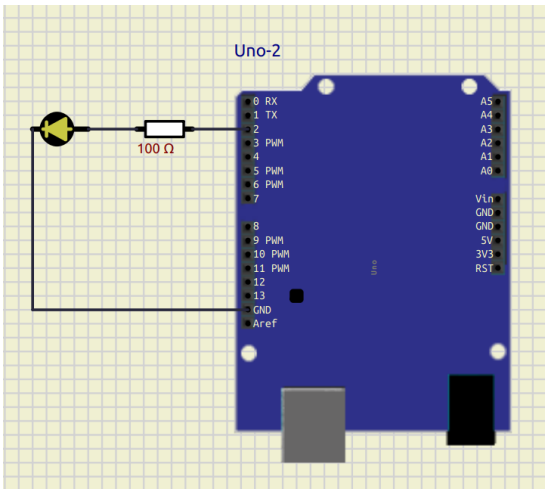
Sommaire

TP Noté : Création d’un capteur de temperature et d’humidité....1
Partie 3 : Simulide....3
3.1.1 Clignotement de led....3
3.1.2 Bargraphe....5
3.1.3 : Capteur DHT22....6
3.1.4 : LCD I2C....7
3.1.5 : LCD I2C DHT22....9
Partie 5 :MQTT....10
5.2.4 : Tests MQTT....10
Partie 6 : Programmer le protocole MQTT....11
6.1.3....11
6.2.1 Partie Wireshark....11
6.2.2 QOS , message retenu et characterristiques génériques....14
Questions....14
Partie 7 : IHM CLI....17
Partie 8 : IHM Web....18
Architecture du projet :....19
Fonctionnalités :....20
Endpoints HTTP du Server Web :....24
Base de données....26
Axes d'amélioration....27

Partie 3 : Simulide

3.1.1 Clignotement de led

Capture d’écran de la simulation :



Code :

```
#define PIN_LED 2

void setup() {
  // put your setup code here, to run once:
  pinMode(PIN_LED, OUTPUT);
}

void loop() {
  // put your main code here, to run repeatedly:
  digitalWrite(PIN_LED, HIGH);
  delay(500);
  digitalWrite(PIN_LED, LOW);
  delay(500);
}

```

Partie electronnique : Pin utilisé : Pin 2, Pin output numérique (0V,5V) car une led pour un clignotement n’a pas besoin de valeur analogiue (PWM)

Problème recontrés :

Compilateur non présent dans Simulide,

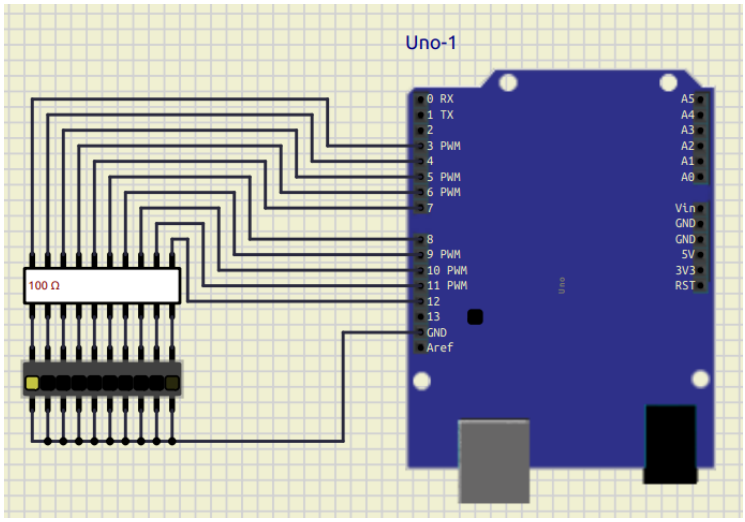
Solution Trouvée : Compiler via l’IDE Arduino, exporter le binaire compilé, importer le firmware

DEL à l’envers

Solution Trouvée : Retourner la DEL

3.1.2 Bargraphe

Capture d’écran de la simulation :



Code :

```
#define LastBargraphPin 12
#define FirstBargraphPin 3

int iterator = 3;

void setup() {
  // put your setup code here, to run once:
  //Pins du bargraph de 3 à 12
  for(int i = FirstBargraphPin;i <= LastBargraphPin;i++){
    pinMode(i,OUTPUT);
  }
}

void loop() {
  // put your main code here, to run repeatedly:
  digitalWrite(iterator , HIGH);
  if(iterator==FirstBargraphPin){
    //si le pin à activer est le 3 le pin 12 doit ^etre dés
    digitalWrite(LastBargraphPin , LOW);
  }else{
    digitalWrite(iterator-1 , LOW);
  }
  if(iterator >= LastBargraphPin){
    iterator = FirstBargraphPin;
  }else{
    iterator++;
  }
  delay(100);
}
```

Pins utilisés : pins de 3 à 12 en output numérique.

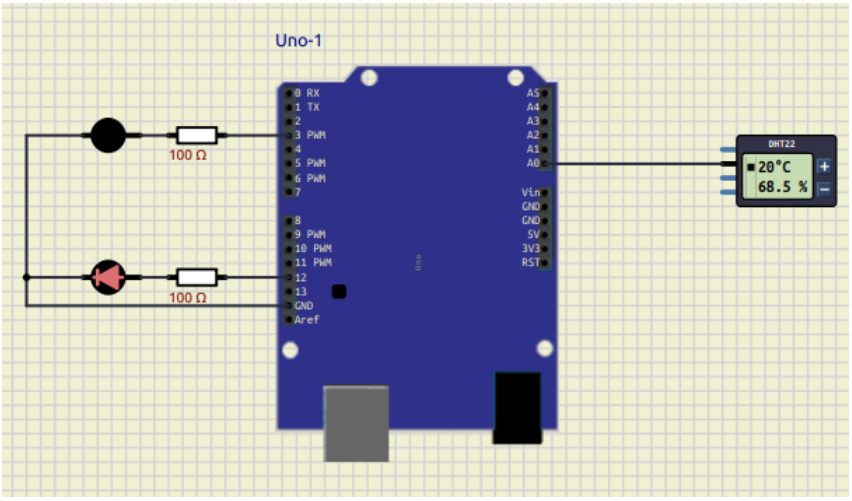
Problème recontrés :

Variable globale non initialisables dans le setup,

Solution Trouvée : initialiser la variable globale hors des fonctions

3.1.3 : Capteur DHT22

Capture d’écran de la simulation :



Code :

```ino
#include "DHT.h"

//Sensor PIN
#define DHT22_DATA A0

//LED Pins
#define RED 12
#define GREEN  3

DHT dht(DHT22_DATA, DHT22);


void setup() {
  dht.begin();
  pinMode(RED, OUTPUT);
  pinMode(GREEN, OUTPUT);
}

void loop() {
  //float h = dht.readHumidity();
  float t = dht.readTemperature();

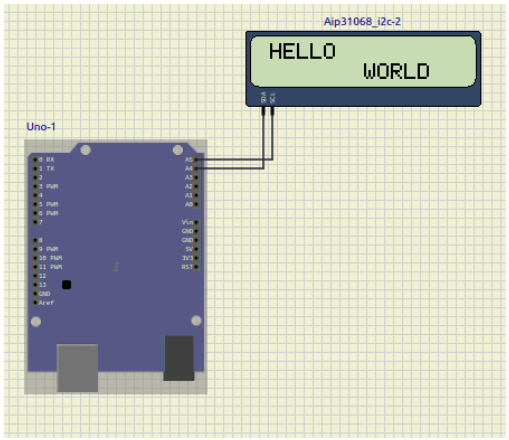
  //t in °C
  if (t < 20.0) {
    digitalWrite(RED, LOW);
    digitalWrite(GREEN, HIGH);
  }else{
    digitalWrite(RED, HIGH);
    digitalWrite(GREEN, LOW);
  }

}
```

Pins utilisés : pins de 3 et 12 en output numérique pour les LEDs Verte et Rouge et pin A0 pour le capteur

3.1.4 : LCD I2C

Capture d’écran de la simulation :



Code :

```
#include <LiquidCrystal_AIP31068_I2C.h>

// set the LCD address to 0x3E for a 16 chars and 2 line display
LiquidCrystal_AIP31068_I2C lcd(0x3E,16,2);

void setup()
{
  // initialize the lcd
  lcd.init();
  // Print a message to the LCD.
  lcd.setCursor(1,0);
  lcd.print("HELLO");
  lcd.setCursor(8,1);
  lcd.print("WORLD");
}


void loop()
{
}
```

Protocole Utilisé : I2C avec les pins

SDA de l’écran connecté au pin A5 de l’arduino

SCL de l’écran connecté au pin A4 de l’arduino

Problème recontrés :

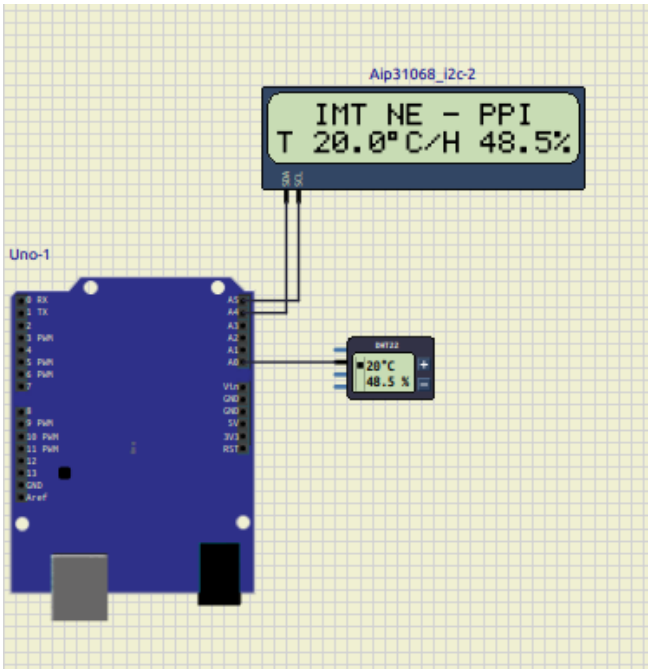
documentation d’une autre librérie donné

Solution Trouvée : documentation de la bonne librairie à utiliser :

https://github.com/red-scorp/LiquidCrystal_AIP31068/blob/master/examples/HelloWorld_I2C/HelloWorld_I2C.pde

3.1.5 : LCD I2C DHT22

Capture d’écran de la simulation :



Code :

```
#include <LiquidCrystal_AIP31068_I2C.h>
#include "DHT.h"
// set the LCD address to 0x3E for a 16 chars and 2 line display
LiquidCrystal_AIP31068_I2C lcd(0x3E,16,2);
//Sensor PIN
#define DHT22_DATA A0

DHT dht(DHT22_DATA, DHT22);

float old_t = 0;
float old_h = 0;

void setup()
{
  dht.begin();
  // initialize the lcd
  lcd.init();
  // Print a message to the LCD.
  lcd.setCursor(2,0);
  lcd.print("IMT NE - PPI");
  lcd.setCursor(0,1);
  lcd.print("T tt.toC/H hh.h%");
  lcd.setCursor(6,1);
  //impression du charactère "°"
  lcd.print((char)223);
}

void loop()
{
  float h = dht.readHumidity();
  float t = dht.readTemperature();

  if(h!=old_t ){
    old_t=t;
    lcd.setCursor(2,1);
    lcd.print(t,1);//ici le 1 veux dire 1 chiffre àaprès la virgule
  }
  if(h!=old_h ){
    old_h=h;
    lcd.setCursor(11,1);
    lcd.print(h,1);
  }
}
```

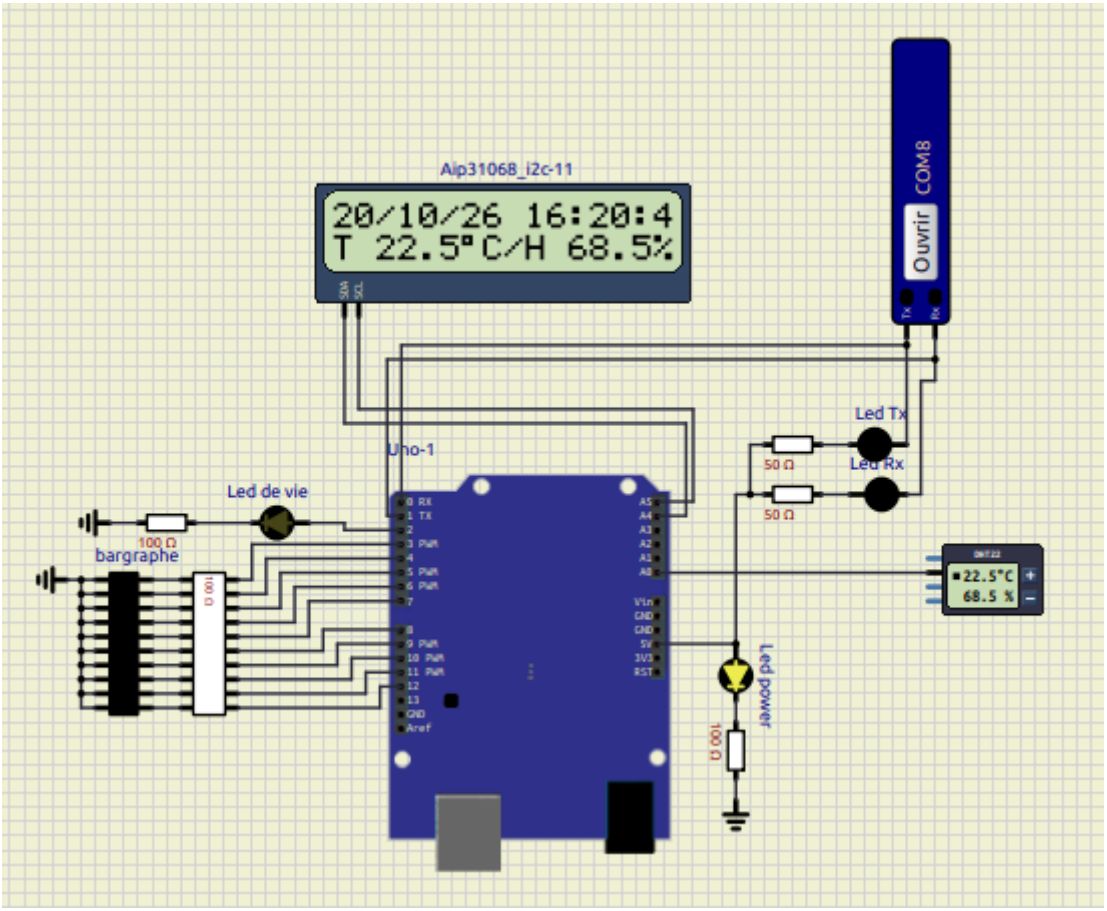
Fonctionnalités : affichage uniquement des informations modifiés

Question : Que pouvez-vous conclure par rapport aux differnets circuits déjà réalisés

ces circuits sont des parties du capteur de tempreratures et doivent se connecter ensemble pour réaliser le capteur final

3.2.1 : Circuit TempHumiX

Capture d’écran de la simulation :



Partie 5 :MQTT

5.2.4 : Tests MQTT

- suite à la souscription à une variable d’un autre poste nous voyons les messages envoyés à l’autre poste via le topic

- après avoir rajouté le topic « TESTS/# » nous voyons tous les messages des topics

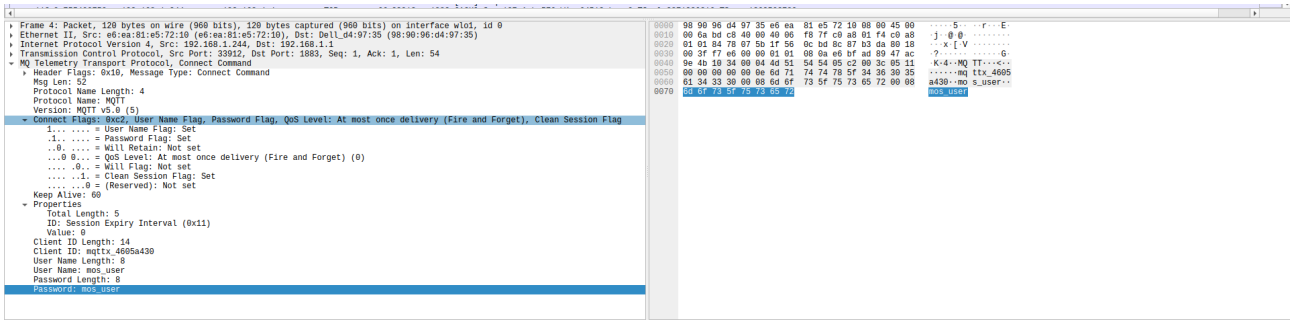
- En envoyant les messages En retain et en désactivant et reactivant le subscriber ils sont renvoyés à la connexion (le retain garde en mémoire jusquà que quelqun l’ai reçu (niveau brocker))

Partie 6 : Programmer le protocole MQTT

6.1.3

l’arduino ne se connecte pas car il ne prends pas en charge le WIFI 5GHz maisn uniquement le WIFI 2.4GHz

6.2.1 Partie Wireshark



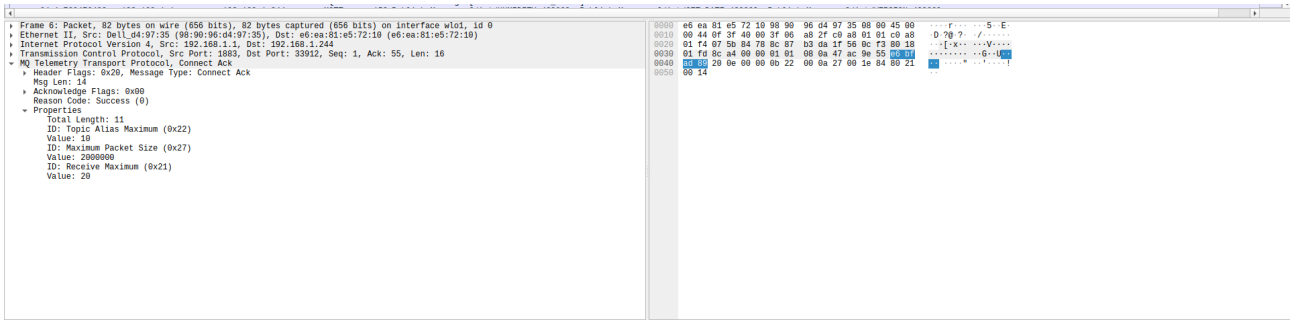
Message de demande de connexion :*

client id:Mqttx_4605a430

Keep_Alive:60

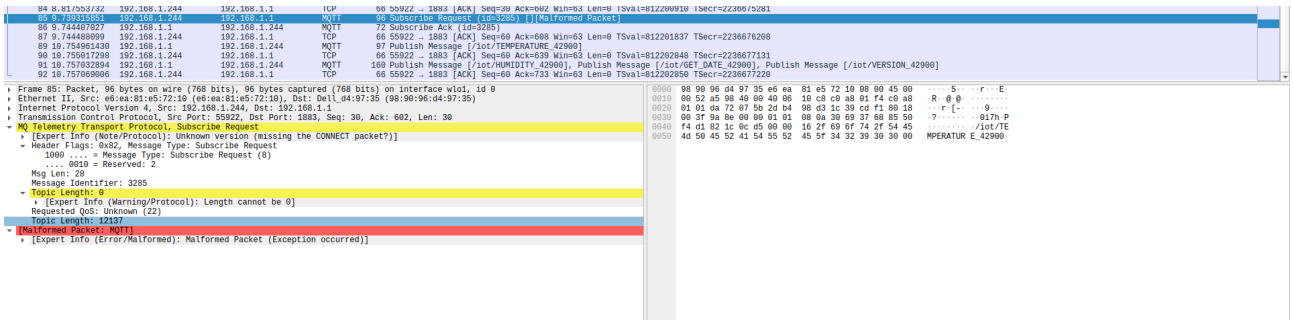
UserName : mos_user

Password : mos_user



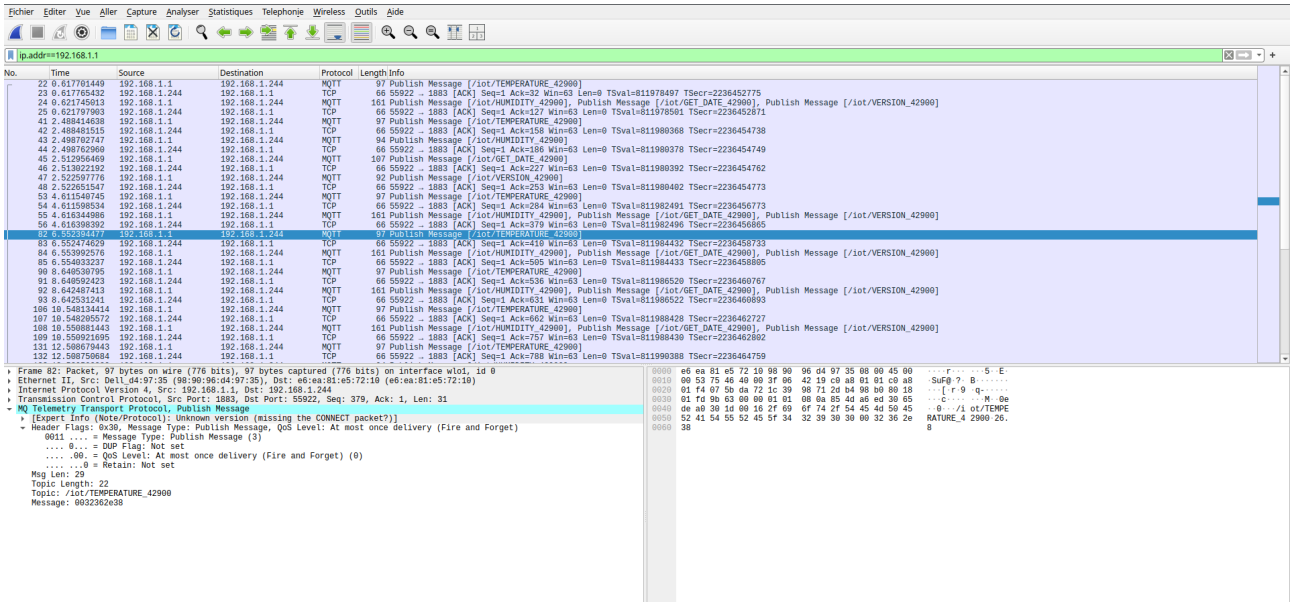
message de validation de la connexion :

Reason code: Success (0)



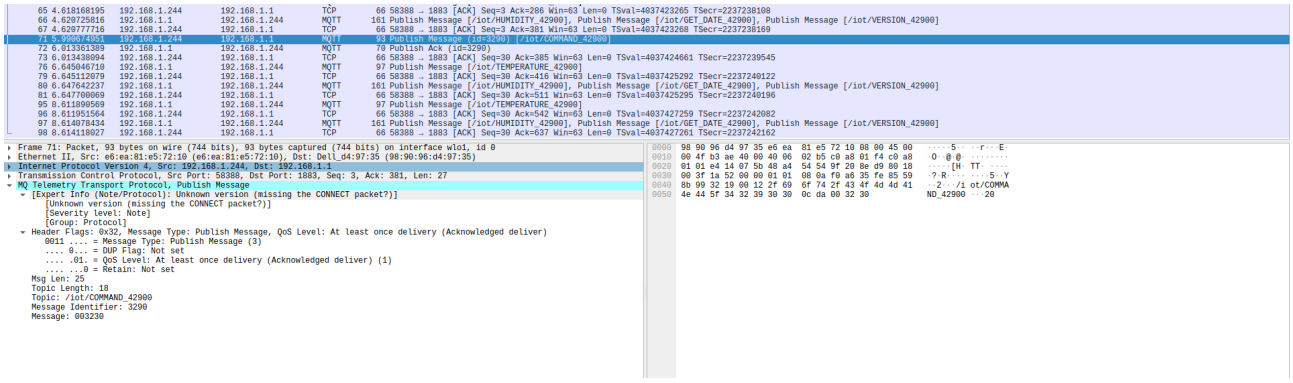
meessage de demande de souscription : Type de message : Subscribe Request (8) Topic : /iot/TEMPERATURE_42900 QOS :Unknown (22)

suivi d’un message de validation SubscribeACK (avec la Reason : Granted QOS 0 (0))



message MQTT Reçu:longeur de topic :22 Topic :/iot/TEMPERATURE_42900 message 26.8

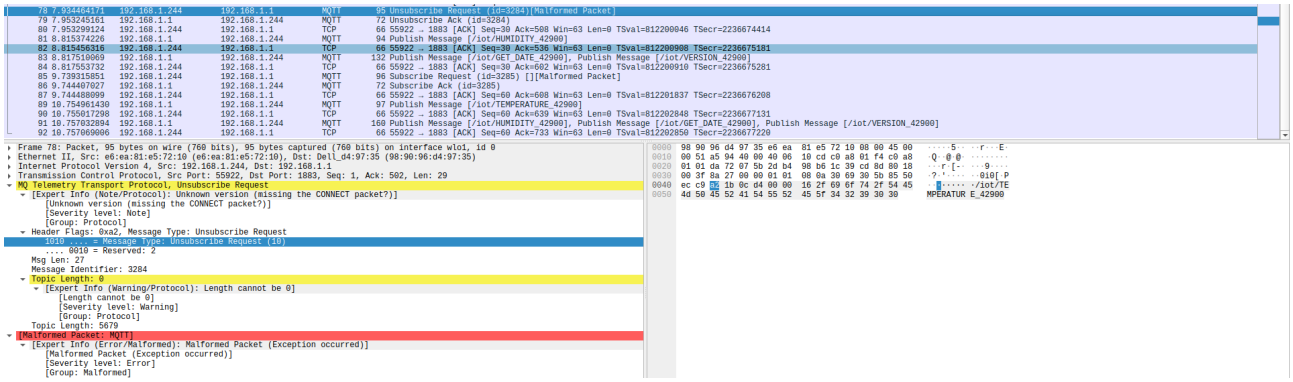
Publish :



message MQTT envoyé :Type de message:Publish message Topic :/iot/COMMAND_42900 message : 20 QOS : at least one 1 retain false

suivi d’un message bublish ack (id du publish : 3290)

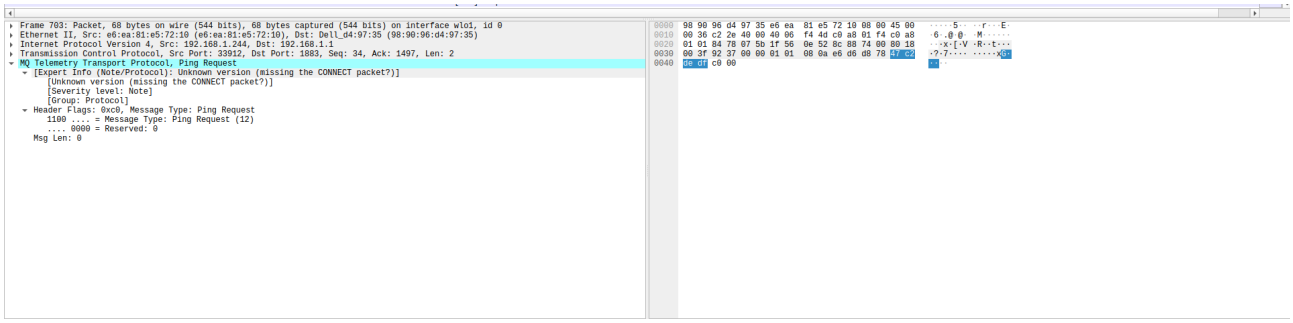
Déconnexion :



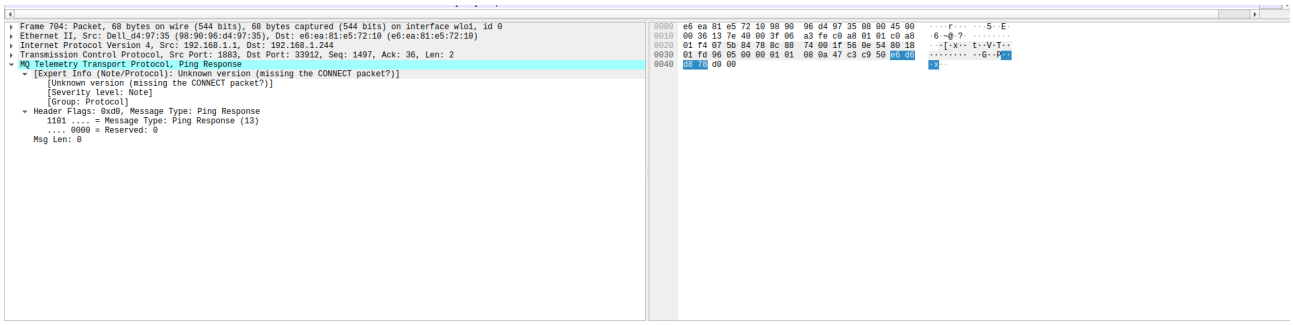
meessage de demande de souscription : Type de message : UnSubscribe Request (8) Topic : /iot/TEMPERATURE_42900

suivi d’un message de validation Unsubscribe ACK (id du unsubscribe 3285)

Ping :



Ping request : aucune donnés, sens : client vers brocker



Ping Response aucune donnés, sens : brocker vers client

Le mot de passe apparait en plain text dans le paquet connect, les echanges ne sont pas chiffrés

un paquet publish de temperature est de 776 bits un paquet HTTP serait bien plus grand

le champ keep alive permet de détecter la déconnexion non volontaire entre l’appareil et le brocker. Si le delai depuiss le dernnier message entre l’appareil et le borcker épasse la valleur d’un keep alive le client effectue une reqête ping avec le brocker qui lui renvoie la reqête

6.2.2 QOS , message retenu et characterristiques génériques

No.	Time	Source	Destination	Protocol	Length	Info
35	2.934717390	192.168.1.244	192.168.1.1	MQTT	93	Publish Message [TESTS/Test_42900]
36	2.952828204	192.168.1.1	192.168.1.244	MQTT	93	Publish Message [TESTS/Test_42900]

message de QOS 0 : un message pour le publish et un message pour la reception

66	10.259989272	192.168.1.244	192.168.1.1	MQTT	95	Publish Message (id=17850) [TESTS/Test_42900]
67	10.280397799	192.168.1.1	192.168.1.244	MQTT	93	Publish Message [TESTS/Test_42900]
69	10.281487997	192.168.1.1	192.168.1.244	MQTT	70	Publish Ack (id=17850)

message de QOS 1 : un message de publish et deux messages pour la réception : (un publish et un publish ack)

120	15.597428315	192.168.1.244	192.168.1.1	MQTT	95	Publish Message (id=17851) [TESTS/Test_42900]
121	15.599185787	192.168.1.1	192.168.1.244	MQTT	70	Publish Received (id=17851)
123	15.608973702	192.168.1.244	192.168.1.1	MQTT	71	Publish Release (id=17851)
124	15.610776211	192.168.1.1	192.168.1.244	MQTT	93	Publish Message [TESTS/Test_42900]
128	15.652978769	192.168.1.1	192.168.1.244	MQTT	70	Publish Complete (id=17851)

message de QOS 2 : un message de publish et quatre messages pour la réception : (un message publish receved(du brocker) un message publish released (du client) un message publish message (du brocker)et un message publish complete (du brocker))

Retain :

80	11.140146933	192.168.1.244	192.168.1.1	MQTT	100	Publish Message [TESTS/Test_42900]
99	15.229193342	192.168.1.244	192.168.1.1	MQTT	90	Subscribe Request (id=17855) [][Malformed Packet]
101	15.248652322	192.168.1.1	192.168.1.244	MQTT	72	Subscribe Ack (id=17855)
103	15.249792201	192.168.1.1	192.168.1.244	MQTT	100	Publish Message [TESTS/Test_42900]

lors de l’envoi avec retain le message est gardé en memoire et renvoyé par le brocker après le subscribe au topic ici le message a été envoyé pui le subscribe fait et ensuite le message a été recu (ici QOS 0)

les topics TESTS/+ et TESTS/# permettent d’avoir accès à tous les topics commencant par TESTS/ et $SYS/# permet s’accéder à tous les messagezs systèmes du brocker (ids messages publiés connexions au brocker, uptime...)

Questions

1.

e réseau IoT est constitué des capteurs, du routeur Teltonika RUTX50 et du broker MQTT. Lorsqu'un capteur effectue une mesure, il publie les données sur un topic MQTT. Le broker reçoit ces messages et les transmet aux clients abonnés aux topics correspondants.

L'acheminement des informations peut être expliqué avec les couches du modèle OSI :

1. Couche 1 – Physique : les données circulent sous forme de signaux électriques ou radio, notamment grâce à la connexion Wi-Fi.
2. Couche 2 – Liaison de données : le protocole Wi-Fi (IEEE 802.11) permet les échanges entre les équipements d'un même réseau local.
3. Couche 3 – Réseau : le protocole IP permet d'adresser les équipements et au routeur Teltonika d'acheminer les paquets vers leur destination.

4. Couche 4 – Transport : le protocole TCP assure une transmission fiable des données entre le capteur et le broker, notamment via le port MQTT 1883.
5. Couche 5 – Session : la gestion de la connexion MQTT permet de maintenir les échanges entre les clients et le broker.
6. Couche 6 – Présentation : les données sont représentées et encodées dans un format compréhensible par les applications.
7. Couche 7 – Application : le protocole MQTT organise les échanges grâce aux messages `PUBLISH`, `SUBSCRIBE` et aux topics associés.

Le routeur assure donc la connexion entre les équipements et l'acheminement des paquets IP, tandis que le broker MQTT centralise la réception et la distribution des messages. Les données peuvent ensuite être récupérées par l'application MQTTX

2.

Quels services de couche applicative offre le TELTONIKA RUTX50 ?

Le RUTX50 propose plusieurs services de couche applicative adaptés aux communications IoT et industrielles. Il prend notamment en charge MQTT pour l'échange de messages, Modbus pour communiquer avec les automates industriels, HTTP/HTTPS pour les échanges Web et SNMP pour la supervision réseau. Il propose également des fonctionnalités OPC UA sous certaines conditions.

Dans une architecture IoT, où situeriez-vous le RUTX50 (Cloud, Fog, Edge) ?

Le RUTX50 se situe principalement au niveau Edge, en tant que passerelle IoT industrielle. Il fait le lien entre les équipements de terrain (capteurs, automates, machines) et les plateformes Cloud. Il assure la collecte et la transmission des données, ainsi que certains traitements locaux. Il peut donc également remplir quelques fonctions associées au Fog.

Quels rôles peut-il jouer dans l'interconnexion des réseaux (pont, passerelle...) ?

Le RUTX50 peut jouer le rôle de routeur IP pour interconnecter des réseaux Ethernet, Wi-Fi et 5G. Il peut également fonctionner comme une passerelle réseau pour permettre aux équipements locaux de communiquer avec des réseaux distants, ou comme un pont (Bridge) selon la configuration. Enfin, il peut assurer une fonction de passerelle IoT en facilitant les échanges entre protocoles industriels comme Modbus et applicatifs comme MQTT.

3.

Le TELTONIKA RUTX50 est équipé d'un microprocesseur (MPU) ARM Cortex-A7 quadricœur cadencé à 717 MHz, accompagné de 256 Mo de RAM et de 256 Mo de mémoire Flash. Il fonctionne sous RutOS, un système d'exploitation basé sur Linux et OpenWrt. Cette architecture lui permet d'exécuter plusieurs services réseau et applicatifs, ce qui en fait une véritable passerelle IoT programmable, et non un simple routeur matériel.

4.

les connexions WIFI du RUTX 50 sont limités à 150 utilisateurs. Le risque est que une personne se connecte dessus et bloque un des utilisateurs la toposogie utilisée est une topologie en étoile

5.

Les dernières versions du firmware RutOS du Teltonika RUTX50 intègrent plusieurs mesures de sécurité : mises à jour correctives, chiffrement HTTPS, renforcement de l’authentification, verrouillage après plusieurs tentatives de connexion échouées, protection de la mémoire (ASLR et PIE) et filtrage des connexions grâce au pare-feu. Ces dispositifs permettent de limiter les intrusions et de protéger les communications du routeur.

6.Analyse des logs de démarage du brocker

la version du brocker utilisé est la 2.1.2

le fichier de config est dans /mosquitto/config/mosquitto.conf

le bridge,la persistance,TLS,TLS-PSK, les websockets sont supoortés

aucun message n’a été restoré

l’autiaentifaiction a été recu en basic

quelques warnings sur les permissions

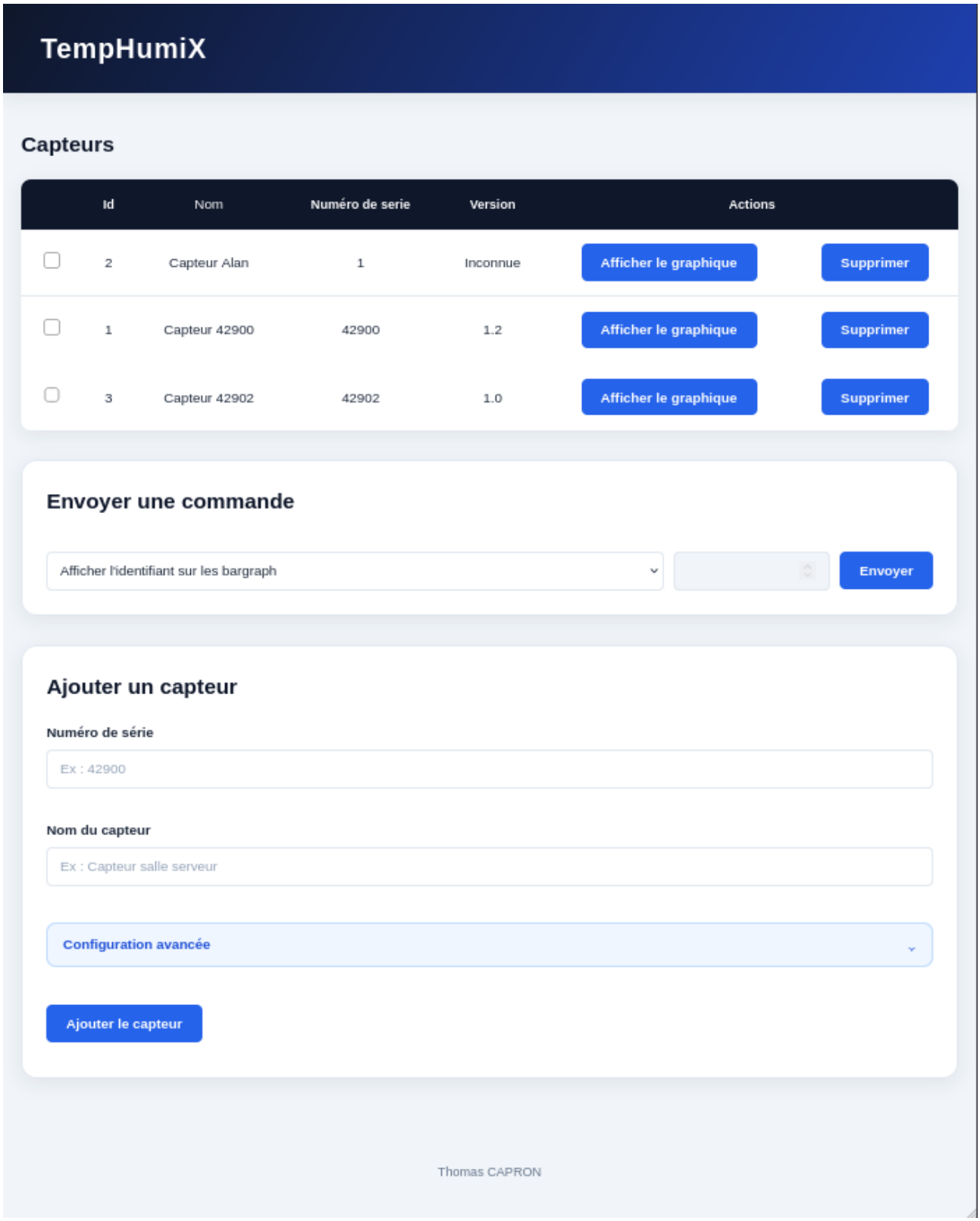
le port IPV4 et IPV6 1883 écoute et est activé

Partie 7 : IHM CLI

Trois programmes ont étés développés :

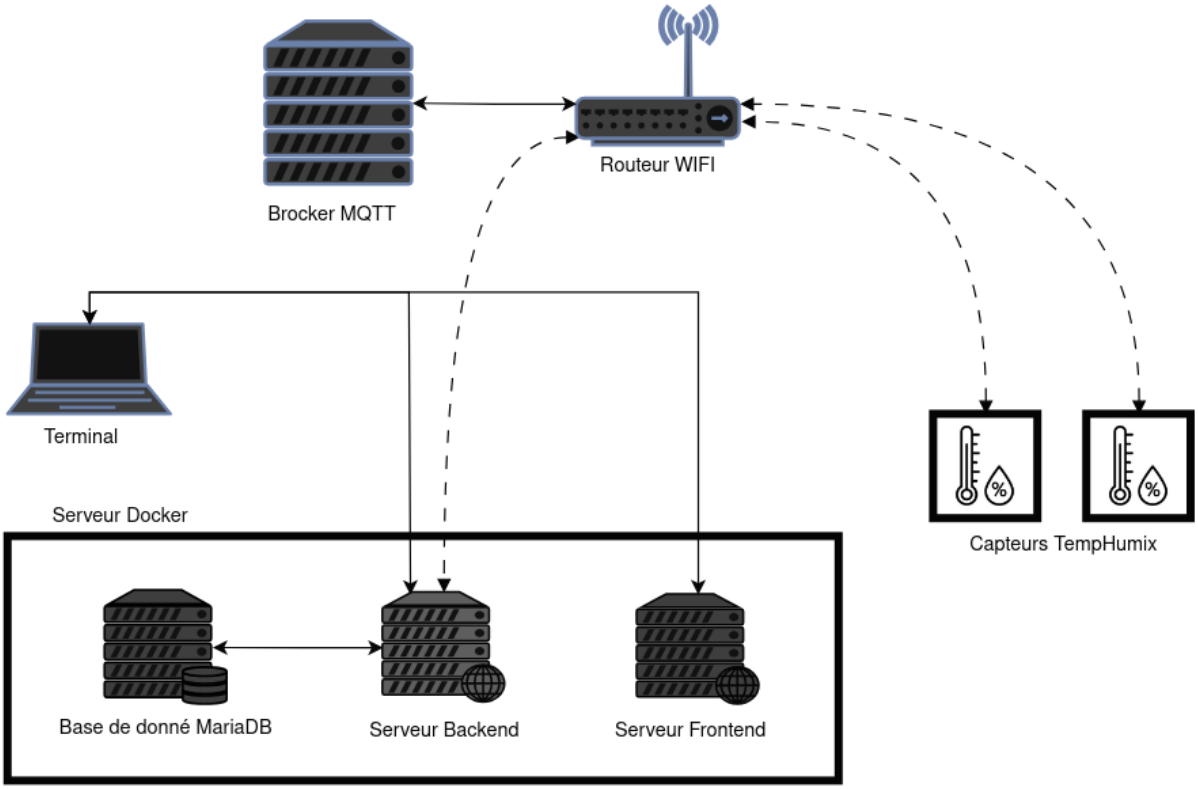
- console_mqtt_ecriture : qui permet d’envoyer via le terminal des commandes au capteur
- console_mqtt_lecture : qui permet de lire les messages MQTT envoyés par le capteur
- thread_mqtt.py : qui permet d’envoyer les commandes au capteur et de lire les messages MQTT envoyés par le capteur

Partie 8 : IHM Web



Capture d’écran de l’interface Web

Architecture du projet :



L'IHM est divisé en 3 conteneurs Docker,

- temphumix-back : conteneur Python hébergeant le backend FastAPI.

  Bibliothèques python utilisées :

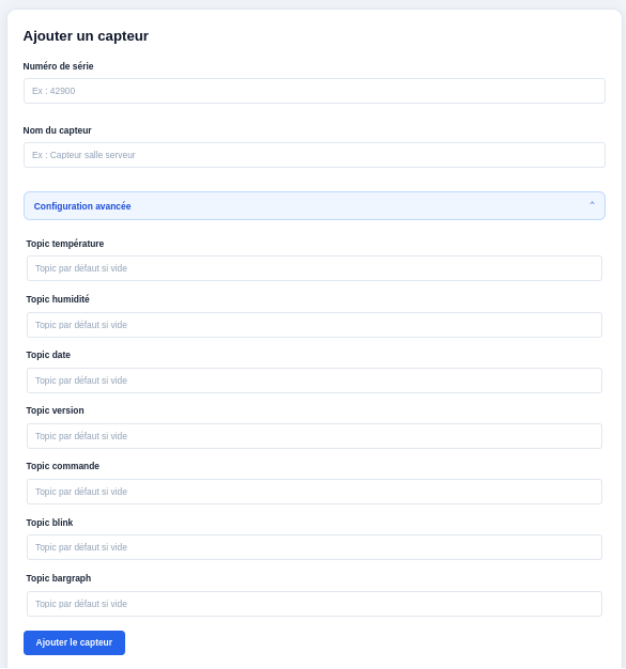
  ```
  fastapi             # Gestion des endpoints REST
  uvicorn[standard]   # Serveur ASGI pour FastAPI
  paho-mqtt           # Communication avec le broker MQTT
  python-dotenv       # Gestion des variables d'environnement
  sqlalchemy          # Gestion de la base de données sous forme d'objets
  pymysql             # Connexion à MariaDB
  ```

- temphumix-front : conteneur Nginx hébergeant les fichiers HTML, CSS et JavaScript.
- temphumix-db : conteneur MariaDB hébergeant la base de données TempHumiX.

  Les données sont conservées dans un volume Docker afin de les préserver lors de la recréation du conteneur.

Fonctionnalités :

1. Ajout d’un capteur



Description :

L’utilisateur peut ajouter un capteur à la liste à l’aide du formulaire. Il doit renseigner son numéro de série ainsi qu’un nom permettant de l’identifier.

La section « Configuration avancée » permet de personnaliser les différents topics MQTT du capteur. Lorsqu’un topic n’est pas renseigné, il est généré automatiquement à partir du numéro de série selon les formats suivants :

/iot/TEMPERATURE_{serialnumber}

/iot/HUMIDITY_{serialnumber}

/iot/VERSION_{serialnumber}

/iot/GET_DATE_{serialnumber}

/iot/COMMAND_{serialnumber}

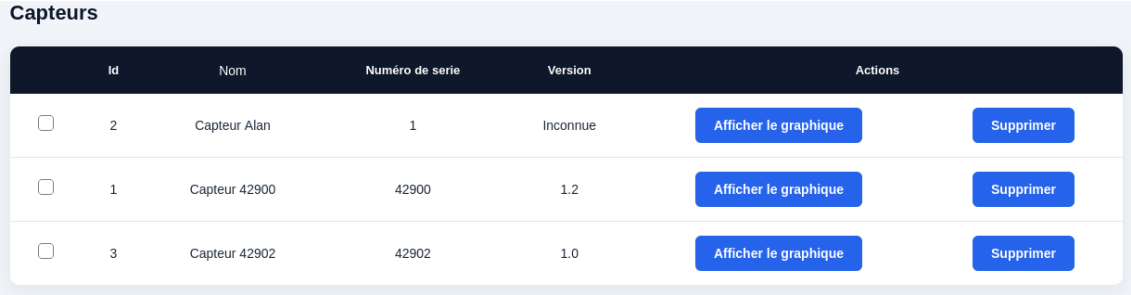
/iot/BLINK_{serialnumber}

/iot/BARGRAPH_{serialnumber}

Endpoint utilsé:

POST /api/sensors

2. visualisation des capteurs :



Description :

Les capteurs enregistrés sont affichés dans un tableau situé en haut de la page.

Chaque ligne contient l'identifiant, le nom, le numéro de série et la version du logiciel du capteur.

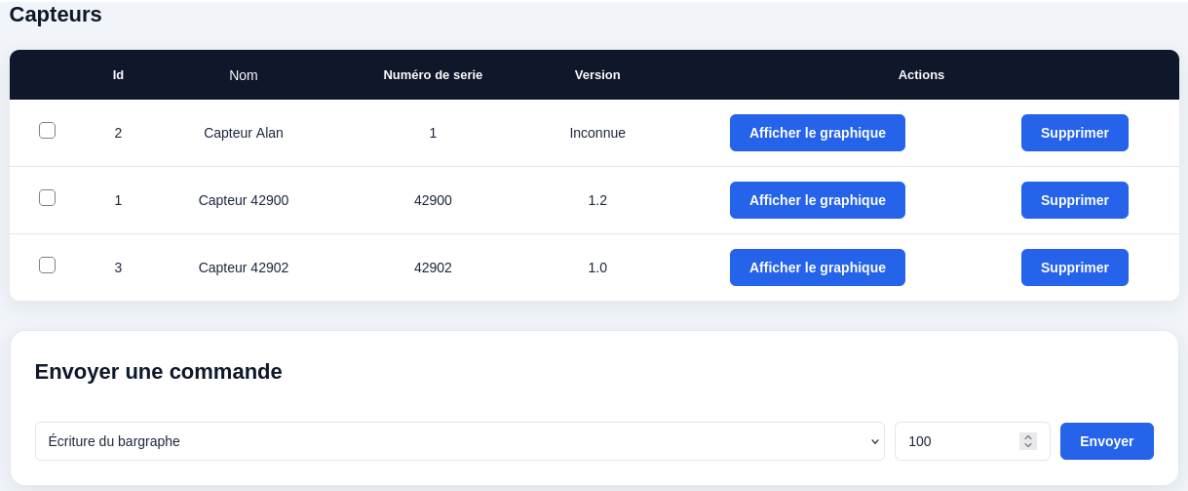
Un bouton situé à la fin de chaque ligne permet de supprimer le capteur correspondant.

Endpoints utilisés :

GET /api/sensors

DELETE /api/sensors/{serialnumber}

3. Envoi d'une commande à un ou plusieurs capteurs



Description :

l'utilisateur peut sélectionner un ou plusieurs capteurs, choisir une commande et renseigner une valeur lorsque celle-ci est nécessaire.

Les commandes disponibles sont les suivantes :

Mise à l'heure du capteur.

Démarrage des mesures.

Arrêt des mesures.

Clignotement lent de la LED.

Clignotement rapide de la LED.

Clignotement personnalisé (une valeur de délai supérieure à 0 est nécessaire).

Écriture du bargraphe (la valeur doit être comprise entre 0 et 1023).

Affichage de l'identifiant sur le bargraphe. Cette fonctionnalité, ajoutée par rapport au TP, permet de distinguer visuellement les capteurs sélectionnés.

Si plusieurs capteurs sont sélectionnés, la commande est envoyée à chacun d'entre eux.

Endpoint utilsé :

POST /api/send-command

4. Visualiser les donnés via un graphique



Description :

Les relevés des capteurs sont enregistrés dans la base de données. Ils peuvent être visualisés sous forme graphique en cliquant sur le bouton « Afficher le graphique » dans le tableau des capteurs.

Le graphique représente la température et l'humidité. Il est actualisé automatiquement toutes les 10 secondes.

L'utilisateur peut sélectionner une période à l'aide d'une heure de début et d'une heure de fin.

Si seule l'heure de début est renseignée, l'heure actuelle est utilisée comme heure de fin.

Si seule l'heure de fin est renseignée, le graphique affiche les mesures de l'heure précédente jusqu'à cette heure.

Si aucune date n’est renseignée, le graphique affiche les mesures de la dernière heure.

Endpoint utilisé :

GET /api/sensors/{serialnumber}/readings

Endpoints HTTP du Server Web :

- GET /api/sensors

  Récupère la liste des capteurs enregistrés dans la base de données.

  Exemple de sortie :

  {"status":"success","count":3,"sensors":
  [{"id":1,"serialnumber":"42900","name":"Capteur
  42900","topic_t":"/iot/TEMPERATURE_42900","topic_h":"/iot/HUMIDITY_42900
  ","topic_date":"/iot/GET_DATE_42900","topic_version":"/iot/
  VERSION_42900","topic_command":"/iot/COMMAND_42900","topic_blink":"/
  iot/BLINK_42900","topic_bargraph":"/iot/
  BARGRAPH_42900","version":"1.2","autoimported":false}]}

- POST /api/sensors

  Ajoute un capteur dans la base de données.

  Exemple d’entrée :

  {"serialnumber":"2","name":"capteur Test","TempTopic":"topic
  tmp","HumiTopic":"topic hum","dateTopic":"topic date","versTopic":"topic
  vers","CommandTopic":"topic comm","BlinkTopic":"topic
  blink","BargraphTopic":"topic bar"}

  Si un topic n’est pas renseigné, il est généré automatiquement.

  Exemple de sortie :

  {"status":"success","message":"Capteur ajouté avec succès","sensor":
  {"serialnumber":"2","name":"capteur Test","topic_t":"topic tmp","topic_h":"topic
  hum","topic_date":"topic date","topic_version":"topic vers","topic_command":"topic
  comm","topic_blink":"topic blink","topic_bargraph":"topic
  bar","autoimported":false}}

- DELETE /api/sensors/{serialnumber}

  Supprime le capteur correspondant au numéro de série fourni dans l’URL.

  Exemple de sortie pour le numéro de série 2 :

  {"status":"success","message":"Capteur supprimé","serialnumber":"2"}

- POST /api/send-command

  Envoie une commande aux capteurs sélectionnés à partir des identifiants fournis.

  Exemple d’entrée : affichage de la valeur 100 sur les bargraphes des capteurs dont les identifiants sont 1, 42900 et 42902.

  {"capteurs":["1","42900","42902"],"commande":"7","value":"100"}

Résultat : réponse HTTP 200 OK lorsque la requête est traitée avec succès.

- GET /api/db-test

  Vérifie l’état de la connexion à la base de données MariaDB.

  Exemple de sortie :

  ```
  {"status":"connected","database":"TempHumiX","version":"11.4.13-MariaDB-ubu2404","tables":["conf","reading","sensor"]}
  ```

- GET /api/sensors/{serialnumber}/readings

  Récupère les relevés de température et d’humidité d’un capteur.

  Exemple d’entrée (paramètres de période facultatifs) :

  ```
  /api/sensors/1/readings?heure_debut=2026-11-04T09%3A50%3A00&heure_fin=2026-11-04T13%3A50%3A00
  ```

  Exemple de sortie :

  ```
  {"serialnumber":"1","heure_debut":"2026-11-04 09:50:00","heure_fin":"2026-11-04 13:50:00","temperature":[{"time":"2026-11-04 12:37:14","reading":"22.80"}],"humidity":[{"time":"2026-11-04 12:37:14","reading":"47.90"}]}
  ```

Base de données

La base de données utilisée est MariaDB. Elle contient trois tables :

- **sensor** : stocke les informations des capteurs enregistrés, notamment leur numéro de série, leur nom, leur identifiant et leurs topics MQTT.
- **reading** : stocke l'historique des mesures reçues par les capteurs. Chaque relevé contient le numéro de série, le type de mesure (température ou humidité), sa valeur et sa date.
- **conf** : permet de stocker des paramètres généraux sous forme de clés et de valeurs. Cette table est présente dans la base, mais n'est pas utilisée actuellement.

La connexion entre Python et MariaDB est réalisée à l'aide des bibliothèques SQLAlchemy et PyMySQL. SQLAlchemy facilite les opérations de lecture et d'écriture en manipulant les tables sous forme d'objets Python.

Axes d'amélioration

Plusieurs améliorations pourraient être apportées à l'IHM Web afin de rendre le système TempHumiX plus autonome et plus pratique à utiliser. Parmi les évolutions envisageables :

- Détection automatique des capteurs : ajouter automatiquement dans la base de données les nouveaux capteurs détectés grâce à leurs messages MQTT, sans devoir les enregistrer manuellement.
- Surveillance de l'état des capteurs : afficher si un capteur est connecté ou non, en fonction de la réception de ses derniers messages MQTT.
- Amélioration des graphiques : permettre de comparer les mesures de plusieurs capteurs sur deux graphiques séparés (un pour l’humidité et un pour la température).

Ces améliorations permettraient de faciliter la gestion des capteurs et d'améliorer l'exploitation des données collectées.